UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI

CORSO DI LAUREA IN INFORMATICA

Improving Estimation of Polarization in Online Discourse

Advisor:
Prof. Elena CASIRAGHI

Thesis by:
Michele DE CILLIS
Student ID: 24260A

Co-Advisor:
Prof. Michele COSCIA

Academic Year 2024-2025

"Somewhere, something incredible is waiting to be known."

Carl Sagan

Table of Contents

Figures

Tables

Algorithms

Code

Introduction

In this thesis, we will delve into the field of network science and examine the properties and peculiarities of networks. First, Chapter 2 provides a broad introduction to network science, graph theory, and polarization.

Next, in Chapter 3, we introduce the tools and the initial dataset used throughout the research internship.

In Chapter 4, we present the starting point of the project, beginning with a short introduction and discussing the limitations that provided an opportunity for enhancement.

In Chapter 5, we report the actual work we have done, which is then presented and discussed in Chapter 6.

Lastly, Chapter 7 summarizes our contributions and discusses the open questions that remain unaddressed.

1.1. NERDS: Network, Data and Society

NERDS[1] is the research group where I carried out my internship. It is an interdisciplinary research group that studies Network Science, Artificial Intelligence (AI), and Computational Social Science (CSS). The environment is also interdisciplinary: it includes students, PhD candidates, PostDocs and professors with backgrounds in physics, computer science, mathematics and sociology. It is located in Copenhagen, within the IT-Universitetet i København (ITU). Research interests include, among others: science of science, social networks, complex networks, urban sustainability, urban and human mobility, data visualization and foundational aspects of complex systems.

1.2. Computational Social Science

Computational Social Science (CSS) is a discipline that studies classical social sciences (sociology, anthropology, economics and political science) using modern tools to explore them with innovative and large-scale approaches.

CSS uses two main approaches: an *empirical* one, which leverages big data to generalize problems and provide analyses and inferences useful for research, and a *scientific* one, which allows building models and simulations of certain phenomena. Moreover, in recent years, thanks to the explosion of artificial intelligence, tools like Natural Language Processing (NLP) or the more recent Large Language Models (LLMs) have accelerated research because of their ability to annotate data with higher accuracy than a non-expert human; consequently, it became possible to automate tasks that would otherwise require a large amount of time[2] or money[3]. In the following chapters, we will provide a concrete example

[1] https://nerds.itu.dk/

[2] Manually annotating millions of data points can compromise the feasibility of a project [1].

of automatic annotations via NLP models first and LLMs later, which were very useful in the project to classify message toxicity and political stance.

In this thesis we use an empirical CSS approach; we will analyze data from a social network (Reddit) to determine its ideological polarization. In the following chapters, we will therefore introduce the concept of polarization (from a sociological perspective) and Network Science, a field that studies complex networks.

1.3. Objectives of the thesis

The initial work performs an extensive analysis on political subreddits (i.e., communities; we will discuss them in detail in the next chapter) on Reddit, showing how they changed over the years, the distribution of Democrats and Republicans, and how users' opinions changed over time on seven topics (*abortion*, *climate change*, *gender identity*, *gun control*, *healthcare*, *racism* and *immigration*), which have become increasingly divisive in the U.S. public debate. It also analyzes ideological and affective polarization in this social network, phenomena that led Reddit users to interact only with like-minded users.
This study is focused on the U.S. context because more than 50% of daily site visitors are from the United States and we have a large amount of data available.

A network is formed by a set of nodes, representing users, and a set of edges, indicating that two users had a significant interaction.

The limitation of the initial project was representing networks as undirected, thus losing information about directionality. My work fits into an extension of the initial project: supporting directed networks and, subsequently, analyzing the new results to verify whether they are, first, reliable and consistent with expectations and, second, useful to better understand the original network by providing previously unavailable information.
My work focuses primarily on computing polarization, using a new method that allows computing the Laplacian matrix on directed networks.

In summary, we want to understand whether it makes sense to add complexity by supporting directed networks, or if undirected networks already provide a satisfactory approximation.

In Network Science, simple (undirected) networks are often preferred because adapting all measures to directed networks is complicated and sometimes not possible. The final goal is to add one more piece to the larger puzzle of generalizing and understanding complex systems.

[3]Services such as Amazon Mechanical Turk (https://www.mturk.com/) can be costly for labs with limited funds.

State of the Art

2.1. Social Network Analysis

Social Network Analysis (SNA) is not a strict theory but rather a general strategy to analyse social structures. It was born long before computing, within sociology, with the aim of studying people's behaviour according to the context they inhabit. Relationships between the "actors" in a network are the priority; nonetheless, individual properties of an actor are necessary to analyse social phenomena.

Thanks to large-scale diffusion of technology and growing computer performance, SNA found a synergy with computer science. Currently, SNA focuses on studying social networks such as Facebook, Twitter (X) and Reddit, among the main ones, due to the large amount of available data.
Increasing computational power has aided SNA by providing a mathematical and practical tool to perform analyses. By modelling networks via graph theory (borrowed from algebra), computer science enabled running complex algorithms on large networks with thousands or millions of nodes in relatively little time.

Another aspect of SNA is studying how social structures influence an individual's behaviour. Two types of SNA are distinguished: ego network analysis, where an "ego node" in a network is identified and all properties of the network restricted to adjacent nodes up to a certain radius are studied [2]; and global network analysis, where one does not focus on a single node but instead tries to study all relations among participants in the network.

2.1.1. Polarization

By polarization we mean the tendency of a group to make more divisive and extreme decisions compared to the initial individual opinions of its members. It also refers to the phenomenon where group members strengthen their opinions after having a discussion on a given topic.

It is an important phenomenon in social psychology and is found across many contexts. Polarization began to be discussed in the 1960s with the study of the "risky-shift" [3], i.e., the tendency of a group to make riskier decisions compared to the same individual decisions taken separately by each member.

More recently, the Internet and social media have introduced a new context in which to study polarization. Researchers have shown that social networks can generate episodes of polarization even when people are not physically close.

For the purposes of this thesis and the internship, we will focus specifically on political polarization, a phenomenon where a person's — or a party's — political opinions diverge from the centre toward extreme positions. We can therefore say that there is little or no overlap between the positions of the considered parties. Scholars distinguish between *ideological polarization* (differences in political positions) and *affective polarization*.

Ideological Polarization: the increase in differences between individuals' political positions, and consequently a reduced dialogue.

Affective Polarization: measures a person's aversion to interacting with people holding different political views.

Initially these were measured with surveys [4], where people answered questions about attitudes toward opposing parties and behaviours they would enact toward members of an opposing party (would they be friends? happy to have them as neighbours?). Today, polarization can also be measured by analysing social networks [5] which, unlike surveys, measure actual behaviours at scale.

In this thesis we will analyse and quantify only ideological polarization using the *node vector distance* measure.

2.2. Graph Theory

Graph theory is a branch of mathematics and computer science that models situations or processes as nodes (actors) and edges (interactions between nodes).

A graph is called directed (also *digraph*) when edges connecting nodes have a direction; otherwise it is undirected.

Formally, we define a graph:

$$G = (V, E)$$

where:

- $V = \{V_1, V_2, ...\}$ is a set of nodes
- $E \subseteq \{\{x, y\} \mid x, y \in V \wedge x \neq y\}$ is a set of undirected edges, **or**
- $E \subseteq \{\{x, y\} \mid (x, y) \in V^2 \wedge x \neq y\}$ is a set of directed edges

These can be represented as data structures with an adjacency list or an adjacency matrix. The adjacency list associates to each node i the list of all nodes i' to which there is an edge. It requires $\Theta(V + E)$ memory.

The adjacency matrix M, instead, uses an $N \times N$ matrix where N is the number of nodes:

$$M = \begin{cases} M_{ij} = 0 \Rightarrow \nexists (i, j) \in E \\ M_{ij} = w \Rightarrow \exists (i, j) \in E \end{cases}$$

where w is the weight of edge (i, j). It requires $\Theta(V^2)$ memory.

Graphs are used to model many relationships and processes across fields. In computer science, graphs were fundamental to develop multiuser and multi-process operating systems (resource management) [6] and to expand the Internet globally and continuously (packet routing) [7,8].

2.2.1. Main properties

Given an undirected graph $G = (V, E)$ and a directed graph $G_1 = (V, E_1)$, they present the following properties:

1. **Degree**: The degree of a node is the number of incident edges. For a node $x \in V$, it is defined as:

$$\deg(x)$$

   In digraphs, we distinguish $\deg_{in}(x)$ and $\deg_{out}(x)$, the number of incoming and outgoing edges respectively.

2. **Walk**: A walk is a sequence of nodes $v_1, v_2, ..., v_n$ such that consecutive nodes in the sequence are adjacent:

$$w = \{v_1, v_2, ..., v_n\}$$

3. **Path**: A path is a walk where all nodes in the sequence are distinct:

$$p_1 = \{w_1, w_2 \in w \mid w_1 \neq w_2\}$$

4. **Cycles**: A cycle is a path where the starting and ending node are the same:

$$p_2 = \{(i, e_1), (e_1, e_2), ..., (e_n, i)\}$$

5. **Clique**: A clique is a subset of a graph G such that for every pair of nodes in the subset, there exists an edge connecting them.

$$c = \{i, j \in V \mid (e_i, e_j)\exists \in E \vee (e_j, e_i)\exists \in E\}$$

6. **Connected Components**: A connected component of a graph G is a subgraph in which any pair of nodes is connected by a walk and that is not part of a larger connected subgraph.

7. **Distance**: The distance between two nodes equals the number of edges in a shortest path connecting them.

8. **Diameter**: The diameter of a graph is the *longest shortest path*, i.e., the maximum distance between two nodes.

9. **Trees**: A tree is an undirected graph where every pair of vertices is connected by exactly one path.

10. **Bipartite Graphs**: A graph is bipartite if it can be divided into two disjoint subsets G' and G'', where every edge from G' connects nodes in G''.

11. **Density**: Density indicates how connected a graph is. If every node is connected to all others, the graph is complete. Conversely, a graph with few edges relative to nodes is sparse. For an undirected graph, density is:

$$d = \frac{2 * |E|}{|V|(|V| - 1)}$$

2.3. Network Science

Network Science studies complex networks. It is multidisciplinary with roots in *mathematics* (graph theory), *physics* (statistical mechanics), *statistics* (statistical inference), *sociology* (social structures) and *computer science* (data mining). It is defined as "the study of network representations of physical, biological and social phenomena that lead to the creation of predictive models of such phenomena." [9]

Many complex situations can be modelled as networks:

- **Social Networks**: In computer science this is one of the most recurrent examples. Social networks literally model relationships and interactions among people. It is natural to think of people as nodes and relationships as edges. Instagram or Twitter are examples of directed networks because person a can follow person a', but a' might not reciprocate. Thus there is a directed edge from a to a' but not vice versa;
- **Citations in scientific articles**: when an article is published it contains n citations to other articles and joins the existing articles network. Each article is a node and a citation is an edge. These networks are directed as well;
- **Protein-Protein Interaction**: in biology, protein-protein interactions occur when two or more proteins interact through biochemical reactions. These interactions occur inside cells. Nodes are proteins and edges represent interactions.

Network Science boomed after Barabási-Albert's "Emergence of Scaling in Random Networks" [10]: real large complex networks do not develop randomly (the probability that node a links to node a' is not approximable as random as in the Erdős-Rényi model) [11], but follow a *power-law degree distribution*: new nodes tend to link to already well-connected nodes. This is called *preferential attachment*. Consequently, a few nodes (hubs) have high degree while most nodes have low degree.

Graph theory and Network Science are tightly interconnected. The latter uses graph theory to represent information and run algorithms. For clarity, the following paragraphs focus on properties useful in Network Science.

2.3.1. Degree distribution

Degree distribution is the probability distribution of node degrees. For a network with n nodes, the probability a node has degree k is:

$$P(k) = \frac{n_k}{n}$$

2.3.2. Laplacian matrix

The Laplacian matrix L encodes topological information of a graph. Given an undirected graph $G = (V, E)$ with adjacency matrix A_G and degree matrix D_G, the Laplacian is

$$L_G = D_G - A_G$$

L_G, of size $|V| \times |V|$, is symmetric and the sum of each row and column equals 0:

$$\sum_{i=0 \in |V|} L_{ij} = 0$$

and

$$\sum_{j=0 \in |V|} L_{ji} = 0$$

In a directed graph the Laplacian uses the indegree or outdegree matrix, $D_{G_{in}}$ or $D_{G_{out}}$, so it is not symmetric and some Laplacian properties do not hold. Therefore, it is often symmetrized or the graph is treated as undirected.

A Laplacian always satisfies:

- Symmetry: $L_{ij} = L_{ji}$;
- Positive semidefiniteness: all eigenvalues $\lambda_1, \lambda_2, ...\lambda_n \geq 0$;
- $\lambda_0 = 0$;
- Row sums zero: $\sum_{i=0 \in |V|} L_{ij} = 0$;
- Column sums zero: $\sum_{j=0 \in |V|} L_{ji} = 0$.

The Laplacian has many applications in graph theory and network science. Studying its eigenvalues and eigenvectors enables *spectral analysis*, which provides insights into network structure or community evaluation. It allows computing the *node distance vector*, that is, the propagation of a node property across the network [12]. It also appears in physics for modelling electrical networks [13] and is used to compute the number of spanning trees via Kirchhoff's theorem [14].

Different Laplacian variants exist: the normalized Laplacian compensates for degree inequality; the incidence-based Laplacian is used for edge-weighted networks; and the *magnetic laplacian* represents directed graphs by treating edge directions as phases in the complex plane. We will deepen the magnetic Laplacian later as it is central to this project.

2.3.3. Null Model

A *null model* is a network model used as a benchmark against a real network. It is randomly generated preserving certain properties of the real network (e.g., density, degree distribution, assortativity). It attributes observed behaviour to a restricted set of properties by generating random networks that share them. It can be used to find correlations between properties: given a real network with property X and observed Y, generate random networks with X to test whether Y persists.

A null model can be random or generative [15]. The random model commonly uses rewiring, where edges are reshuffled while preserving node degrees. In

Figure 1 an example is shown. The generative approach starts from a partition of the original network and adds new nodes and edges until predefined null hypotheses are met.

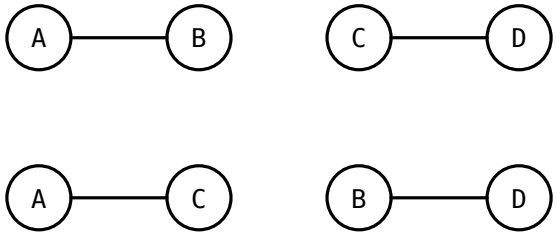


Figure 1: Above: before rewiring. Below: after rewiring

2.3.4. Generative Models of Networks

Generative models of networks are mathematical stochastic models designed to simulate the creation of complex networks. There are different models, depending on the objective and the analysis that one wants to perform. As null models, they are used to test and generalize properties and mechanisms of a real network, on random networks. Among the most known models, we have:

1. **Stochastic Block Model**: is a generative model of graphs that tends to create graphs whose nodes are grouped in communities [16]. Given n the number of nodes, k the number of communities, and a matrix $P = k \times k$ of probabilities. The matrix P represents the probability that two nodes of different communities are connected by an edge;
2. **Barabási-Albert**: is a model that generates scale-free networks [10], with properties similar to real networks, like the *preferential attachment*. The algorithm accepts a parameter n, i.e., the number of nodes, and a parameter m, i.e., the number of edges that connect to an existing node of higher degree;
3. **Erdős-Rényi**: is a model for generating random networks, given a probability p and n nodes, each node connects to another with probability p [11];
4. **Watts-Strogatz**: is a generative model that surpasses the limitations of the Erdős-Rényi model, favoring the generation of hubs, like in the Barabási-Albert model. Every node has a very small average shortest path. Given a parameter $\beta \in [0, 1]$ and k the number of neighbors of each node, situated in a ring topology, every edge is redirected towards another node with probability β [17];

2.3.5. Backboning

Real networks are full of noise, i.e., edges and nodes that do not have statistical significance and that can pollute the results. It is necessary, therefore, to use a method to remove the noise from the network and keep only the significant elements. This technique is called *backboning*. It arises from the need to keep only the structures and hierarchies relevant in a network, so that it is easier to analyse and more computationally economical.

With backboning we keep a global focus, to highlight the *highways* of a network, i.e., those paths that are important for the flow of information. The backboning has a function similar to the *Principal Component Analysis* in statistics.

There are various backboning algorithms, depending on the phenomena that one wants to highlight and then analyse. Some algorithms consist of: finding the *minimum spanning tree* [18], using a *disparity filter* [18], deriving the *salience skeleton* [19] or the method of *noise correction* [20]. In this thesis we will focus on the last method, as it was used in this project.

The Noise-Correction (NC) algorithm [20] is based on the assumption that every edge is an interaction between nodes. If an edge is to be kept, it must reach or exceed a certain threshold of statistical significance.

Given a graph $G = (V, E, N)$, where V is the set of nodes, E is the set of edges and N is the set of weights and $N \subseteq \mathbb{R}$, with $N_{i.} = \sum_{i \in E} N_{ij}$ and $N_{.j} = \sum_{i \in E} N_{ij}$ and therefore $N_{..} = \sum_{i,j \in E} N_{ij}$, the NC algorithm defines a measure called *lift* L_{ij}, which represents how much the weight of an edge deviates from the expected value of a random model:

$$L_{ij} = \frac{\hat{N}_i}{E[N]_{ij}}$$

with $E\big[N_{ij}\big]$ the expected weight for a pair of nodes (i, j):

$$E\big[N_{ij}\big] = \hat{N}_{i.} \frac{\hat{N}_{.j}}{\hat{N}_{..}}$$

L_{ij} measures how much the weight of an edge, between nodes i and j, is high compared to the expected value:

$$L_{ij} = \begin{cases} = 1 \Rightarrow \text{same weight as expected} \\ > 1 \Rightarrow \text{higher weight than expected} \\ > 0 \wedge < 1 \Rightarrow \text{lower weight than expected} \end{cases}$$

Therefore, it is subsequently centered in 0, which we call $\tilde{L}_{ij}$ (this measure is also called *score*).

Successively, the variance is calculated with the delta method, of the values obtained:

$$var\big[\tilde{L}_{ij}\big] = var\big[\hat{N}_{ij}\big] \left(\frac{2\Big(\kappa + \hat{N}_{ij} \frac{d\kappa}{d\hat{N}_{ij}}\Big)}{\Big(k\hat{N}_{ij} + 1\Big)^2} \right)$$

where $var\big[\hat{N}_{ij}\big]$ is the variance of a binomial distribution:

$$var\big[N_{ij}\big] = N_{..} \hat{P}_{ij} \big(1 - \hat{P}_{ij}\big)$$

and κ:

$$\kappa = \frac{1}{E\left[N_{ij}\right]}$$

and:

$$\frac{d\kappa}{d\hat{N}_{ij}} = \frac{1}{\hat{N}_{i.}\hat{N}_{.j}} - \hat{N}_{..}\frac{\hat{N}_{i.} + \hat{N}_{.j}}{\left(\hat{N}_{i.}\hat{N}_{.j}\right)^2}$$

Given that real networks are sparse and it is difficult to estimate with precision $\hat{P}_{ij}$, we assume that $\hat{P}_{ij}$ uses a Bayesian framework that follows a Beta distribution: $\left[n_{ij} + \alpha, n_{..} - n_{ij} + \beta\right]$. Given that also α and β are unknown, we assume that the generation of edge weights assumes a hypergeometric distribution, in which every time the weight of an edge increases by 1 for node n, then we extract and remove a node j from the set of nodes (distributed according to the weight $N_{i.}$ and $N_{.j}$ of each node). Thus, the mean μ and variance σ^2 are defined, respectively:

$$E\left[p_{ij}\right] = E\left[\frac{N_{ij}}{N_{..}}\right] = \frac{1}{N_{..}}\frac{N_{i.}N_{.j}}{N_{..}} = \mu = \frac{\alpha}{\alpha + \beta}$$

and

$$var\left[p_{ij}\right] = \frac{1}{N_{..}^2}\frac{N_{i.}N_{.j}(N_{..} - N_{i.})\left(N_{..} - N_{.j}\right)}{N_{..}^2(N_{..} - 1)} = \sigma^2 = \frac{\alpha\beta}{(\alpha+\beta)^2}(\alpha + \beta + 1)$$

which can thus be solved in α and β. Which allows us to obtain $var\left[\hat{N}_{ij}\right]$ and, therefore, the variance $var\left[\tilde{L}_{ij}\right]$.

Finally, an edge will be kept only if the weight is greater than $\delta\sqrt{var\left[\tilde{L}_{ij}\right]}$, i.e., if it exceeds δ-times the standard deviation. Where δ is a threshold parameter passed to the algorithm. For more information, refer to the original paper [20].

The NC algorithm favors the maintenance of connections between non-central nodes, but on the same level of hierarchy [2].

2.3.6. Spectral Analysis

The Spectral Analysis is the study of the eigenvalues and eigenvectors of the Laplacian matrix of a graph. Given a Laplacian L, we define the eigenvalues λ:

$$\lambda \in \sigma(L) \quad \sigma(L) = \{\lambda \mid \det(L - \lambda I) = 0\}$$

and the eigenvectors v:

$$v \in \ker(L - \lambda I), \quad v \neq 0$$

As defined above, the first eigenvalue λ_0 in a Laplacian is always 0. The others are monotonic increasing:

$$0 = \lambda_0 \leq \lambda_1 \leq ... \leq \lambda_n$$

The spectral analysis is important because it provides insights into the structure of the graph. It can be used for the graph coloring problem [21] or for a low-rank approximation (approximation of the adjacency matrix by a matrix of lower rank) [22]. In addition, the second and third eigenvectors, v_2 and v_3, are used for the visualization of graphs with a simplified and pleasing layout (also v_4 for 3D visualization), as shown by Hall [23].

One of its common uses is the study of the second eigenvalue λ_2, i.e., the Fiedler Value, also called *algebraic connectivity*. To λ_2 are reserved many other uses [22].

The eigenvalues and eigenvectors have the following properties:

1. The vector of all 1s is always an eigenvector of the first eigenvalue λ_0 of L, with value 0;
2. The largest eigenvalue of the adjacency matrix is always between the average degree and the maximum degree of a node in a graph G [22];
3. If G is connected, then $\lambda_1 > \lambda_2$ and the eigenvector v_1 will be positive [22];
4. The multiplicity of 0 as an eigenvalue of L_G is equal to the number of connected components of L_G.
5. The largest eigenvalue of L is at most double the maximum degree in G;
6. $\lambda_n = -\lambda_1$ if and only if G is a bipartite graph [22].

2.3.7. Community Discovery

Studying a network, it is common to want to analyse if a group of nodes forms a community. That is, if these can be grouped and divided based on a common property. In our society, communities are everywhere: people who live in the same city, in the same group of friends or who have the same favourite actor. Who lives in a certain city will certainly have many interactions with people who live in the same city. Conversely, they will have few or none with people in different cities. The intuition is the same for networks and Network Science. Formally, a community is said to be such when there is a high density within the community and sparse interactions with nodes outside of it.

The study and evaluation of communities in a network are called *community discovery*. This practice has many uses. For example, for the *backboning*, where we can identify similar nodes and remove them, leaving only a single "representative" node, to simplify the network, or for grouping and classifying nodes in clusters specific, to test their behaviour under certain conditions of the network (e.g., in advertising and marketing).

Community discovery is a vast field: there are many ways to group nodes in communities and new methods are continuously studied. Indeed, there is no definitive method; it depends on the objective. Generally, is given importance to the performance of the method of community detection and its reliability, measured by similarity to other algorithms.

The first method found for community discovery is called Stochastic Block Model (SBM), with the maximization of the *likelihood function*. Given an SBM, i.e., a model of generating random graphs that contain communities, generated with two parameters p_{in} and p_{out}, which respectively are the probability that a node interacts with a node within the community and that a node connects with a node outside the community (generally, $p_{in} > p_{out}$), we initialize the two parameters to the same values as the initial network. Successively, we define the *likelihood function*:

$$L_{\Theta,A} = \sum_{u,v \in A} l_\theta, A, u, v$$

where

$$l_{\theta,A,u,v} = \begin{cases} \theta_1 - 1 \Rightarrow A_{uv} = 1 (u,v) \in \theta_3 \\ \theta_2 - 1 \Rightarrow A_{uv} = 1 (u,v) \notin \theta_3 \\ -\theta_1 \Rightarrow A_{uv} = 0 \quad (u,v) \in \theta_3 \\ -\theta_2 \Rightarrow A_{uv} = 0 \quad (u,v) \notin \theta_3 \end{cases}$$

We search to maximize the function, in such a way that:

$$\hat{\theta} = \arg_{\theta \in \Theta} \max L_{\theta,A}$$

Finally, if, given an SBM, $p_{out} > p_{in}$, then we can find all the disassortative communities, i.e., of nodes that link only with nodes that *non* are in their community.

This method is equivalent to the method of modularity optimization [24], which we will discuss more later.

Another common method for finding communities in a network is to use the *random walk*, i.e., starting from a random node, explore casually one of its neighbours, and so on, iterating n times. The idea is that when a random walk enters a community, it will stay there for a long time, given the high number of edges within the community. Conversely, the probability that it arrives at a boundary node and then enters another community is very low. Therefore, using the technique of random walk is not the most efficient method. It works better, however, the Infomap method, which has the objective of minimizing the map equation [25], i.e., a codification of a *random walk*.

Initially, the algorithm simulates a normal random walk to calculate the frequencies of visits to nodes. Every time it explores a node, it assigns a sequence of bits encoded with Huffman coding [26]. To save memory and reuse IDs, in a way similar to the roads that repeat in different cities, it starts to group nearby nodes under the same community, assigning a code of a growing number of bits. In this way, in the encoding, when the random walk enters a new community, it signals by writing initially the number of the community and then the number of every node. When it arrives at a boundary node and moves to a new community, it

uses the code `1111`, which signals the jump to a new community. This adds a bit overhead, because in every community there are at least 5 bits more, but the breakeven point is reached quickly. The process is iterated many times, until the length of the random walk encoding is minimized. Given the nature of random walks, it is a non-deterministic algorithm.

Another method of community detection is the *label percolation* (or *label convergence*): starting from a subset of nodes to which are assigned randomly labels, these are propagated to all remaining nodes, until all nodes are labelled. Also this is a non-deterministic algorithm.

Initially, a label is assigned to each node. Successively, iteratively, each node explores the labels of its neighbours and assigns itself the most frequent label; in case of a tie, it chooses one randomly among the most frequent. It continues until convergence, in which every node has the same label as the majority of its neighbours.
The positive aspect of this algorithm is that it is very simple to implement and converges quickly.

In addition, given the non-deterministic nature, multiple iterations of the same algorithm, reveal different community structures, which can be aggregated using the Jaccard similarity index [27].

Finally, community detection can happen both on static networks (*snapshots* at a certain point in time), and on dynamic networks, where we assume that the network changes, nodes are added, edges are removed, and consequently, communities change.

A naïf method of evaluating communities in dynamic networks is to assume that each snapshot is independent in time and search independently on each snapshot. The literature says that the results can be very different. We can therefore resort to a technique called *evolutionary clustering* [28].

With evolutionary clustering, we try to balance two objectives: maximize the quality of the snapshot at time t, which reflects the most recent changes, and minimize the *history cost*, i.e., the distance between the clustering at time t and that at time $t-1$.

The algorithm uses an index of similarity or a matrix of distances of various timestamps T, constructed over time, defined as M_t. At each timestamp, the algorithm searches to optimize the quality of the snapshot:

$$sq(C_t, M_t) - \alpha \cdot hc(C_{t-1}, C_t)$$

where C_t is the clustering calculated at time t. sq is a function that evaluates the quality of the snapshot, hc is the function of history cost and α is a parameter of trade-off that establishes how much importance to give to the configurations of the previous snapshots.

2.3.8. Modularity

The modularity is a measure that evaluates the quality of a *community evaluation* in a network. A high degree of modularity means that there is a high density within the community and a lower density between a node in a community and one outside. It represents the internal density of the communities. It also has the purpose of optimizing the function of division in communities, with the objective of maximizing the modularity. Given A the adjacency matrix and δ the function delta of Kronecker, which returns 1 if the nodes are in the same community and 0 otherwise, the modularity is defined by:

$$M = \frac{1}{2|E|} \sum_{i,j \in V} [A_{ij} - \frac{\deg(i)\deg(j)}{2|E|}]\delta(c_i, c_j)$$

The domain of modularity is defined in $[-0.5, +1]$: the lower the value, the more disassortativity in the network. Conversely, if it tends to $+1$, the division of communities is optimal. If the modularità is equal to 0, then the graph has no community structure.

2.3.9. Other properties

1. **Omophily ed Heterophily**: The omophily is a qualitative property that expresses how nodes in a network tend to be close to each other when expressing similar features. It is one of the foundations of community discovery, because it is based on the sociological assumption that people tend to relate with people similar (same gender, similar age, similar passions or interests) and is reused in the study of networks because it is assumed that nodes with similar features tend to be connected. This happens both for behavioural reasons (the user in a social network searches only for people or pages that respect his interests) and for environmental reasons (the algorithm of a social network shows more prominently posts that might interest him). The heterophily, on the other hand, is the exact opposite.

2. **Assortativity and Disassortativity**: The assortativity is a quantitative measure to represent the omophily. Conversely, the disassortativity represents quantitatively the heterophily. The intuition of assortativity is that we take numerical features and, given a connection between two nodes, estimate their similarity (e.g., two nodes with values 1 and 5 are more similar than two nodes with values 1 and 5000). The property most common is the degree of two nodes, in which we assume that two nodes with a very high degree are connected. For example, in social networks, it is more likely that a celebrity connects to another.

3. **Coefficient of Clustering**: It is a property that measures how, in a network, nodes tend to be connected to each other. Especially in social networks, nodes tend to have a high density of connections. It is a local or global measure. At the local level, it measures how likely it is that the neighbours of a node form a clique. Given N the set of neighbours of i and $k_i = |N|$:

$$CC_i = \frac{|\{e_{jk} : v_j, v_k \in N, e_{kj} \in E\}|}{k_v(k_v - 1)}$$

At the global level, instead, we base on triplets of nodes (triangles or triads). The global coefficient of clustering represents how many closed triangles there are, relative to all the triads in a network:

$$CC = \frac{\#\text{ triangles}}{\#\text{ triads}}$$

2.3.10. Node Vector Distance

Finding the distance between two nodes is a problem widely studied in the literature [7,8]. Thanks to this, graph theory has been extended to networks like we commonly understand, i.e., an ensemble of computers connected to each other in a way that allows communication. This has enabled the development of the internet, which allows us to exchange data with computers that are on the other side of the world.

However, this approach considers only one aspect: transporting a data from a node i to a node j. In a binary way, a data can be found in a node or in another, or in an intermediate one, but cannot diffuse continuously, with a part of the information present simultaneously in multiple nodes, nor start from multiple nodes at the same time.
In contrast, many real-world situations follow this schema, such as the *diffusion of a virus*, the *quality of a viral marketing campaign* or the *polarization in a social network*: phenomena that start from one or more nodes and spread towards the nodes around.

The intuition behind the *Node Vector Distance* (NVD) is to measure, given a network and two instants t_1 and t_2, the diffusion of a property of a node over time.

Formally [29], given a network non-directed $G = (V, E)$, where V is the set of nodes and E is the set of edges, we define the property A in each node of the network via a vector A of length $|V|$ and domain in $[0, 1]$. Assuming that the network does not change between t_1 and t_2, the NVD measures the distance travelled and the diffusion of the property A over time, as illustrated in Figure 2a and Figure 2b.

(a) $A_{t1} = [1, 0, 0]$ (b) $A_{t2} = [0.1, 0.25, 0.65]$

Figure 2: Diffusion of property A in graph G.

There are three classes of solutions for the NVD [29]: *Generalized Euclidean*, *Shortest Path* and *Spectral*. We will focus on the first, the *Generalized Euclidean*, as it is the one used during the internship.

The *Generalized Euclidean* (GE) measures distances in a network in the same way as they would be measured in an Euclidean multidimensional space. It is defined by:

$$\delta_{A_{t1},A_{t2}} = \sqrt{(A_{t1} - A_{t2})^T L^{\dagger(A_{t1}-A_{t2})}}$$

where $L^\dagger$ is the pseudo-inverse of the Laplacian matrix of Moore-Penrose (the Laplacian is not invertible as it is singular), and $(A_{t1} - A_{t2})^T$ is the transpose of the difference vector of the property A between the two instants considered.

In the literature, this measure has been used to quantify ideological polarization in a social network [30], where the property A represents the political opinion of every user, expressed as a continuous value from -1 to $+1$ (from democratic to republican), and decomposed into A^+ and A^-.

Respectively, $A^+ = \begin{cases} a_i \Rightarrow a_i \geq 0 \\ 0 \Rightarrow a_i < 0 \end{cases}$ and $A^- = \begin{cases} |a_i| \Rightarrow a_i < 0 \\ 0 \Rightarrow a_i \geq 0 \end{cases}$, so that the polarization is given by:

$$\delta_A = \sqrt{(A^+ - A^-)^T L^\dagger (A^+ - A^-)}$$

This measure represents the distance between two nodes selected randomly, weighted by the extremities of their opinions. Consequently, a network composed of nodes with moderate opinions ($0 \leq \overline{A} \leq |0.1|$) will have a lower polarization than the same network with nodes having extreme opinions ($\overline{A} \geq |0.8|$).

2.4. Magnetic Laplacian

The *Magnetic Laplacian*, has roots in physics. Analogous to the magnetic Schrödinger operator [31], it was created to model the behaviour of particles in an electromagnetic field, incorporating a phase factor that rotates the classical Laplacian matrix [32].

By analogy, the phase complex associated with the particles can be interpreted as the direction of the edges in a graph.

Given a directed graph $G = (V, E)$, we define $w_s(i, j)$ as the symmetrized weight of the edge from node i to node j:

$$w_s(i, j) = \frac{w(i, j) + w(j, i)}{2}$$

The phase is given by $e^{i\theta a(i,j)}$, where θ is a parameter and $a(i, j)$ is defined as:

$$a(i, j) = \begin{cases} 1 \Rightarrow (i, j) \in E \\ -1 \Rightarrow (j, i) \in E \\ 0 \Rightarrow (i, j) \wedge (j, i) \in E \end{cases}$$

We define in addition $\psi(i) : V \to \mathbb{C}$, a function that associates a complex number to each node. More specifically, in the context of the paper, it is an eigenfunction,

defined as $\chi_{\theta,i}$, which is the eigenvector associated with the lowest eigenvalue, also said, it minimises the following problem: $\min_{\chi} \langle \chi \mid \hat{\mathcal{L}}_{a,\theta,\chi} \rangle$ s.t. $\langle \chi \mid \chi \rangle = 1$.

The operator $\hat{\mathcal{L}}_{a,\theta}$ is then defined as [33]:

$$\hat{\mathcal{L}}_{a,\theta}\psi(i) = \sum_{j} w_s(i,j)\big(\psi(i) - e^{i\theta a(i,j)}\psi(j)\big)$$

The behaviour of the Laplacian depends on the parameter θ, which represents the electric charge of the particle. For $\theta = 0$, we obtain the classical Laplacian defined in previous chapters: $L = D - A = \hat{\mathcal{L}}_{a,0} = \hat{\mathcal{L}}_{0,\theta}$. The same dynamics occur when $\theta = \theta + 2\pi$, therefore the parameter θ can be interpreted as an angle [33].

The operator, has been found to be closely correlated to the topology of the network. Due to its physics roots, where the electromagnetic field is associated to the unitary Gauge Group $U(1)$[4], the cycles are the uncovered structures. By performing an Hodge decomposition[5] of the edge flow, the components of of the cycles will be considered. Because of the directed nature of G, the cirected cycles are emphasized.

That said, as θ varies, different dynamics are uncovered. Thus, the Laplacian highlights different structures in the network, as described in Table 1.

$\boldsymbol{\theta}$	**Highlighted Structure**
0	Same as classical Laplacian
$\frac{\pi}{2}$	2, 4, 3-cycles
$\frac{2}{3}\pi$	2, 3-cycles
$\frac{4}{5}\pi$	3-cycles
π	Signed Laplacian $a(i,j) = 0 \Rightarrow +$ $a(i,j) \in \{-1, 1\} \Rightarrow -$

Table 1: Comparison of the structures highlighted by the Magnetic Laplacian as θ varies.

The Magnetic Laplacian has found use in community evaluation for directed graphs [33], in spectral analysis [34] and, as we will see in the following chapters, in the calculation of the Node Vector Distance on directed graphs.

[4]A Gauge Group describe a simmetry in physics where, without changing the misurable outcome, the mathematical description of a system is changed. In our case, means that structural properties of the network are kept even by adding edges or cycles.

[5]It's a mathematical tool which decompose a network into high order structures, like cycles or gradients. It is used a different declination of the Laplacian, called Hedge Laplacian.

For example, given a simple directed graph like in Figure 3, calculating the classical Laplacian matrix is equivalent to treating the graph as undirected, obtaining:

$$\begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$$

The magnetic Laplacian matrix with $\theta = \frac{\pi}{2}$, instead, is:

$$\begin{pmatrix} 1 & -\frac{1}{2}i & +\frac{1}{2}i \\ \frac{1}{2}i & 1 & -\frac{1}{2}i \\ -\frac{1}{2}i & \frac{1}{2}i & 1 \end{pmatrix}$$

As seen, the Magnetic Laplacian is capable of highlighting communities formed through directed triangles, also known as 3-cycles, which are cycles involving three nodes, as illustrated in Figure 3. Because of that, during the internship, the directed triangles will be studied in our real world example and starting dataset. However, more complex cycles structure, like the 4-cycles and their relation with the generalized Euclidean, will be analysed in our toy examples. Those experiments will be explored in the next chapters.

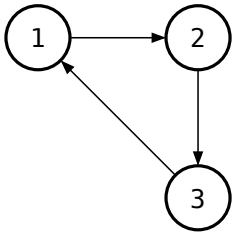


Figure 3: A directed triangle.

Coding the previous expressions with pseudocode, we move from the classical Laplacian (Algorithm 1), to the Magnetic Laplacian (Algorithm 2).

Algorithm 1: Laplacian. Ei = edge_index, Ew = edge_weight

```
function LAPLACIAN(Ei, Ew)
    N ← Length(Ei[1])
    A ← ZERO_MATRIX(N, N)

    for i < N do
        s ← Ei[0][i]
        t ← Ei[1][i]
        A[s][t] ← Ew[i]
        A[t][s] ← Ew[i]
    end

    D ← ZERO_MATRIX(N, N)
    for i < N do
        D[i][i] ← sum(abs(A[i]))
    end
```

```
    return D
end
```

Algorithm 2: Magnetic Laplacian. Ei = edge_index, Ew = edge_weight

```
function MAGNETIC LAPLACIAN(Ei, Ew, θ)
    N ← Length(Ei[1])
    θ_P ← EXP(1j * θ)
    H ← ZERO_MATRIX(N, N)

    for i < N do
        s ← Ei[0][i]
        t ← Ei[1][i]
        H[s][t] ← Ew[i] * θ_P
    end
    H ← ( (H + CONJUGATE_TRANSPOSE(H)) / 2 )

    D ← ZERO_MATRIX(N, N)
    for i < N do
        D[i][i] ← sum(abs(H[i]))
    end
    return D
end
```

Dataset and Development Tools

3.1. Python

Python [35] is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its built-in high-level data structures, together with dynamic typing and dynamic binding, make it very suitable for rapid application development, as well as for use as a scripting or glue language to connect existing components. Python's simple and easy-to-learn syntax emphasizes readability and thereby reduces program maintenance costs.

Python supports modules and packages, which encourage program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form, free of charge, for all major platforms and may be freely distributed.

Thanks to its broad ecosystem of libraries and its fast learning curve, Python is the most used language in Data Science and was used to write the code whose results are described later in this thesis.

3.2. Libraries

3.2.1. NetworkX

NetworkX [36] is a Python library for creating, manipulating, and studying the structure, dynamics, and functions of complex networks and graphs. It provides tools for analyzing the structure and dynamics of social, biological, and infrastructural networks, a standard programming interface, and a graph implementation suitable for many applications, offering a rapid development environment for collaborative and multidisciplinary projects.

It supports algorithm acceleration and additional features via third-party backends and offers an interface to existing numerical algorithms and code written in C, C++, and FORTRAN. With NetworkX you can load and store networks in standard and non-standard formats, generate many types of random and classical networks, analyze network structure, build network models, design new network algorithms, and visualize networks. It is released under a BSD license and is widely used in the scientific network-science community.

3.2.2. NumPy

NumPy [37] is a fundamental Python library for scientific computing, created to support operations and functions on matrices and multidimensional arrays. Released under a modified BSD license, it provides high-level APIs for complex data structures and many efficient mathematical functions.

The main data type is the N-dimensional array (`ndarray`), which enables high-performance vectorized operations. NumPy includes functions for linear algebra, Fourier transforms, random number generation, and many other mathematical operations. Thanks to its C implementation, it offers higher performance than pure Python. It underpins many other scientific Python libraries such as SciPy,

Pandas, and Matplotlib, forming a cornerstone of the Data Science ecosystem. Its intuitive syntax and high performance make it indispensable for numerical analysis and matrix computation.

3.2.3. PyTorch

PyTorch [38] is a Python library for Machine Learning and Deep Learning, providing high-level APIs to build and train neural networks. Originally created by Meta and now part of the Linux Foundation, it is open source and released under a modified BSD license.

The fundamental data type is the tensor, a homogeneous multidimensional array similar to NumPy arrays but with advanced capabilities. PyTorch stands out for native CUDA support, enabling transparent acceleration on NVIDIA GPUs. It offers an eager-execution approach to define computational graphs, which makes debugging more intuitive compared to other frameworks. It includes modules for building neural networks (`torch.nn`), optimizers (`torch.optim`), data handling (`torch.utils.data`), and autograd operations for automatic gradient computation. It is widely used in research and industry for computer vision, natural language processing, and reinforcement learning applications.

3.2.4. Pandas

Pandas [39] is an open-source Python library for data analysis and manipulation. It provides high-level data structures designed for analysis, transformation, and visualization of structured datasets.

Its main structures are Series—one-dimensional arrays with associated indices—and DataFrame—two-dimensional table-like structures with labeled rows and columns. Although internally based on NumPy arrays, they also support non-numeric data such as dates, strings, and categorical types. Pandas offers powerful utilities for loading data from various formats (CSV, Excel, SQL, JSON), data cleaning (handling missing values, duplicates), grouping and aggregation (`groupby`), join and merge operations, and type conversions. Its intuitive API and strong performance make it essential for data wrangling and exploratory analysis in Data Science.

3.3. Reddit

Reddit[6] is a social network where users can post content as links, text, images, or videos, and other users can comment. It is divided into communities called subreddits, prefixed with `r/` (for example `r/politics` or `r/python`), which can be general or topic-specific. With over 100,000 active subreddits, it is considered an aggregator of communities.

The recommendation system works via upvotes and downvotes, votes that registered users can give to posts and comments to influence their visibility.

[6] https://www.reddit.com

Upvotes increase the chance that content will be shown, while downvotes reduce it. This mechanism determines the ordering of content on homepages and within communities.

As of December 2025, Reddit ranks among the top 10 most visited websites worldwide and is the fourth most used social media platform [40].

To construct the networks, a dump of all public Reddit data from its creation until 2025 is downloaded [41].

Overview of the Project

In this chapter we will explain the starting point of the project, describing its characteristics, the libraries used, the steps for building the network and its limitations.

4.1. Starting Point

The project analyses political subreddits over time, from their foundation until today. It examines how user interactions have changed in quantity and quality, such as toxicity, opinion differences and network homophily. The process begins by downloading a dump of all Reddit posts across years, building a network that captures as many interactions as possible (the following paragraphs will detail these steps), and then starting the actual analyses.

A different network is created for each week to visualize and analyse its evolution over time. The underlying assumption is that if two nodes are connected by an edge, then there was a significant interaction between the two users in the considered week.

4.2. Tools Used

4.2.1. NetworkX

NetworkX was used to generate toy examples and to perform specific graph operations, such as finding the largest connected components (LCC). Although it is not the fastest library in terms of performance, it is particularly useful thanks to the numerous utilities already implemented for network manipulation. Moreover, it provides built-in implementations of realistic network models that were used to create the toy examples.

4.2.2. PyTorch

PyTorch was mainly employed for tensor handling. All data are stored as tensors before being converted to CSV format, exploiting the advantages of this data structure in terms of computational efficiency and compatibility with linear algebra operations. The library was also used during the computation of the pseudoinverse of the Laplacian matrix.

4.2.3. NumPy

NumPy was used to a lesser extent, limited to situations requiring arrays and specific operations on multidimensional arrays. The choice of NumPy was motivated by its superior performance, provided by its C implementation of core operations, which is essential for efficient computation on large volumes of data.

4.2.4. Pandas

Pandas was employed exclusively in the final stages of the process, when data representation for debugging purposes was necessary. Thanks to its built-in utilities and APIs, this library greatly simplifies basic statistical operations and

result aggregation. Its ability to easily handle heterogeneous structured data and perform complex join and merge operations makes it indispensable for exploratory data analysis and rapid prototyping.

4.3. Network Construction Procedure

This section can be divided into 6 phases:

1. **Data Filtering**: Starting from one `.csv` file per month, all posts and comments are iterated through, excluding posts since the analysis focuses exclusively on comments. Next, only data belonging to relevant subreddits (i.e., U.S. political subreddits) are kept. Comments written by bots are also removed — that is, Reddit users that produce automatic replies based on certain triggers.

2. **Preliminary Network**: In this step messages are partitioned into weeks. Only messages with a significant length (15 characters in this case) are kept. Only users who exchanged a number of messages greater than or equal to the average are retained (self-loops, i.e., messages where a user replies to themselves, are not counted):

$$|M_u| \geq \frac{\sum_{u \in U} |M|}{|U|}$$

   Then a network is built where each node represents a user and each edge represents a message (user u replies to user u' or vice versa). Consequently, if two users interacted frequently, there will be more edges connecting them. The larger the number of such edges, the higher the weight (the statistical significance) of the relationship between them. Finally, backboning is applied to the network to slim it down and make it more manageable. The goal is to maximize the number of nodes and minimize the number of edges using the Noise-Correction method, which is based on edges and their weights. The Largest Connected Component (LCC) is returned.

   Additionally, to anonymize data and comply with GDPR, a new id is assigned to each user. A global mapping table is maintained to ensure consistency of user ids across weeks and months.

3. **Topic Detection**: For each network and for each message within it, the BERTopic model [42] is used to automatically classify each message with the most appropriate topic. Each preliminary network is split into two subsets for training and classification respectively. Initially, the model is trained with 4096 messages per week. After training, all messages in each network are labelled. Topics are aggregated and manually reviewed, grouped into macro-topics, and non-relevant ones discarded. Finally, each message is assigned one of the following topics:

- *abortion*: Groups themes like abortion, contraceptive methods and reproductive rights in general;
- *climate*: Contains comments about global warming, deforestation, electric vehicles, fossil lobbies, renewable energies, etc.;
- *gender*: Comments about feminism, the gender pay gap, gender identity, LGBTQ+, pronouns, etc.;
- *guns*: Groups themes like gun regulations, gun lobbies, mass shootings, suicides, militias, etc.;
- *health*: Contains comments about healthcare, child healthcare, insurance, drug development, etc.;
- *racial_justice*: Concerns racial justice and law enforcement in a broad sense. Topics include Black Lives Matter, the police in general, defunding requests and arrests.
- *unauthorized_immigration*: Includes topics such as the U.S. border, deportation or children and immigration in the United States. Posts are not only about unauthorized immigration but can also cover broader discussions about Latin American immigrants.

4. **Toxicity**: The toxicity of each message is computed, with a score ranging from 0 (polite and respectful message) to 1 (vulgar message with insults or threats). The Detoxify model [43] with default settings is used.

5. **Stance**: Using an open-source LLM, Llama 3 [44], the political stance expressed in a message is detected. The stance can be labelled as democratic or republican. Being a binary choice, the classification is simpler. An instance of Llama is initialized with the following prompt:

   ```
   You are an expert political scientist. The following message is
   part of the debate on {topic} in the United States. In this debate
   there are two sides. Side D thinks {democratic_opinion}. Side R
   thinks {republican_opinion}. If the message is ambiguous, it belongs
   to side U. Classify the following message as belonging to side D, R,
   or U. You can only reply with one letter between D, R, or U, no other
   answer is acceptable."
   ```

   Each topic will have a prompt with the same structure, adapted to its content. Given the probabilistic nature of LLMs, the tokens `R` and `D` are returned with their respective probabilities. The value -1 is assigned to a democratic opinion and $+1$ to a republican opinion. The final political stance value of the message will be: $p(R) - p(D)$.

6. **Final Network**: As a last step, final networks are created. Networks can be per-topic or complete. While building the network, users and their messages for a week are collected; messages are grouped by topic and then an average of detected opinions is computed for each user. If a user in a week has not written enough significant comments to compute a score for every topic, the problem is solved using two strategies:

- *rolling opinion*: assume the user's opinion during week x is similar to previous weeks ($x-1$, $x-2$, ..., $x-n$); recover all their messages present in the dataset;
- *zombie mode*: if a user has not expressed opinions on a topic, assume their political stance on that topic is similar to their other expressed stances, computed as the average of their available opinions.

The Largest Connected Component (LCC) is returned, as a connected network with the maximum number of nodes is required.

Finally, dimensionality reduction via Principal Component Analysis (PCA) is performed to produce a synthetic value for each user's political orientation. In Figure 4, an example of a final network is shown.

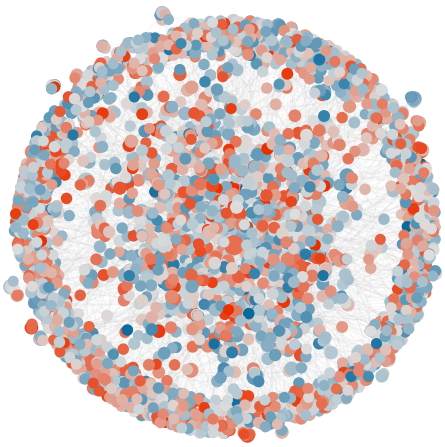


Figure 4: Visualization of an undirected network from the last week of January 2019

4.4. Measuring Polarization

To measure weekly polarization, the network is imported and a tensor is built containing the edge matrix with their attributes and a matrix with node attributes, i.e., each user's political stance for each topic listed above.

The node vector distance is used to compute polarization within the network, as anticipated in Chapter 2.3.10. The Laplacian matrix and its pseudoinverse are then computed, representing the network structure. Finally, the generalized euclidean is calculated, using as parameter the vector containing political stances for a given topic.

4.5. Limit and Research Question

The current project's limitation lies in networks being represented as undirected, which do not capture the direction of interaction between two users. This is not a limitation per se, since the use of undirected graphs in network analyses is widespread. However, numerous scientific articles [45–48] show the limits resulting from the loss of directionality information. Therefore, we wanted to verify whether adding complexity to the project — first by incorporating directionality information and then using algorithms compatible with directed graphs — could produce interesting results, revealing information otherwise hidden in an undirected network.

During the internship, steps 3, 4 and 5 are skipped, because computationally expensive and that would have required the use of a HPC (High Performance Computing), which would have increased the development time. Thus, it has been used a cache file which contain all the messages from the original dataset, their topics classification, its toxicity and stance. This allowed speed up in the iterations between development and tests.

Changes made

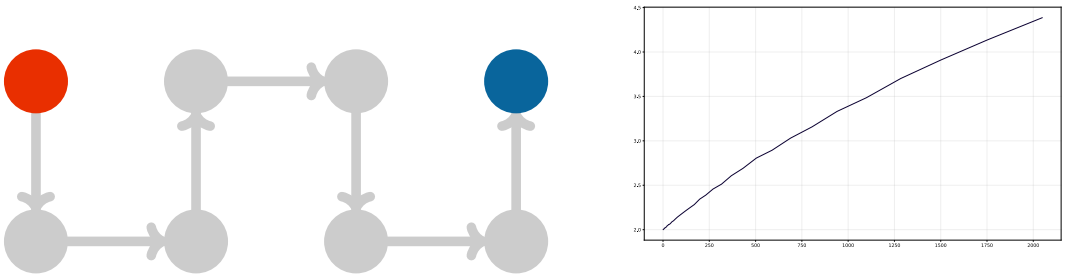
The Magnetic Laplacian, as seen in the previous chapters, is an operator that has found many uses in *community evaluation* and *spectral analysis*. However, until now, it had never been tested in other conditions, such as for solving the *node vector distance*. For this reason, before implementing the operator in the project described in the previous chapter, it was thoroughly tested on toy examples to verify which behaviors it measures most strongly and whether they make sense.

5.1. Toy Examples

We started by testing the operator in some simple graphs, just to ensure that, as with the undirected networks, it would behave in the same way, to grow in conditions in which it is more difficult, for a feature, to spread into the network. This to say, if polarisation grows when the nodes are more extreme and structurally distant.

To do so, we set up a trivial example of growing directional chains graphs (Figure 5(a)), with a growing number of nodes. In the test, they had extreme opinions on the extremes (-1 and +1) and the rest in between are set to 0.

We got a growing polarization (Figure 5(b)) number in the first case, both because of the increasing number of nodes, to which the polarization number is sensitive to [30] and because we actually increase the distance between the only two non-zero valued nodes.



(a) Red: +1, Blue: −1, Gray: 0. (b) x axis is $|V|$, y axis is the polarization value.

Figure 5: (a): Chained Graph. (b): Polarization measure when increasing the number of nodes.

We then also set up more complex models (Stochastic Block Model, Barabási-Albert, Erdős-Rényi and Watts-Strogatz) presented in Chapter 2.3.4, by using the same approach used in [30], performing two experiments: first, we randomly assign moderate opinions to all the nodes and gradually increase the polarization, as shown in Figure 6 and then, we isolate the likely-minded nodes in echo chambers (Figure 7).

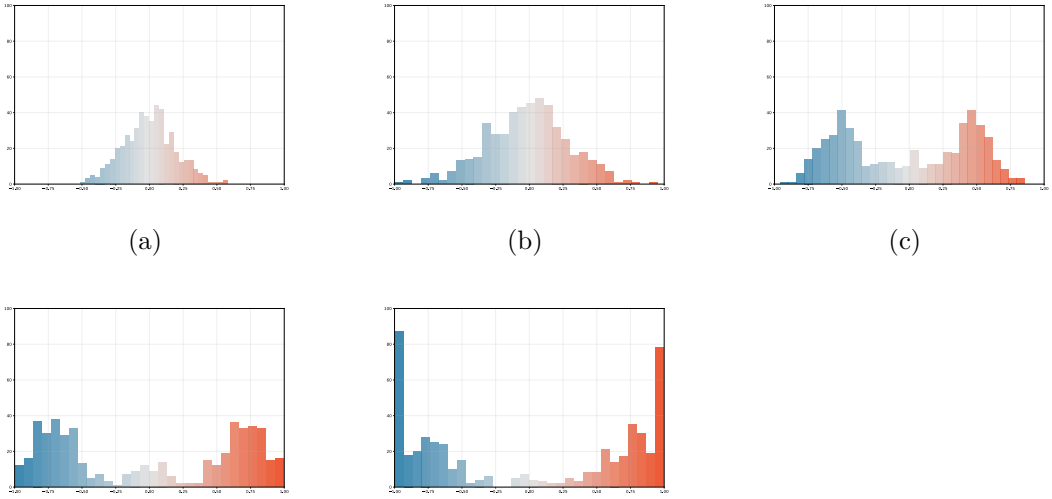
The first experiment (Figure 6), is performed by assigning to the nodes in the networks, values between −1 and +1 with the following distributions:

1. Normal distribution: $\overline{p} = 0$ and $\sigma = 0.20$;
2. Normal distribution: $\overline{p} = 0$ and $\sigma = 0.30$;
3. Bimodal distribution: $\overline{p} = \pm 0.50$, $\sigma = 0.15$;

4. Bimodal distribution: $\overline{p} = \pm 0.70$, $\sigma = 0.15$;
5. Bimodal distribution: $\overline{p} = \pm 0.85$, $\sigma = 0.25$.

Later, in the second experiment (Figure 7), starting from the most extreme network of the previous experiment, we started to isolate the communities ($+1$ and -1 values), by increasingly removing the edges inter-community and, in order to keep nodes-edges ratio, we added intra-community edges, with the following probabilities: 0.2, 0.4, 0.6, 0.75, 0.9.

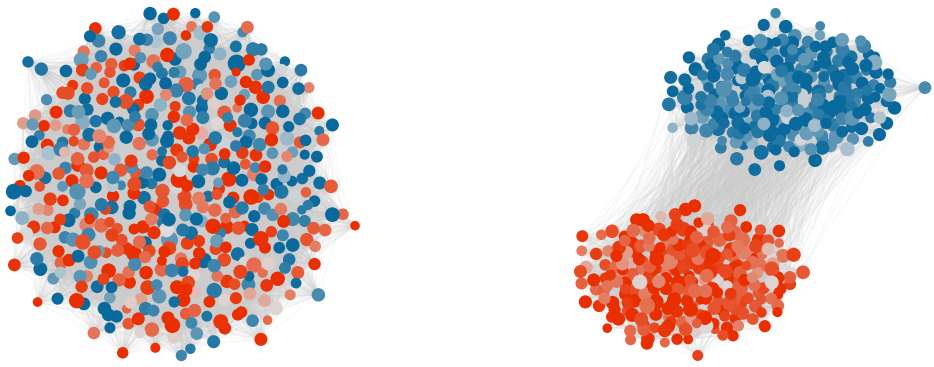
The results, shown in Figure 8, have confirmed to us that the behaviour of the generalized eucliedean, applied to directed networks through the magnetic laplacian, is comparable to his undirected version.



(a) (b) (c)

(d) (e)

Figure 6: Nodes transitioning to more extremes opinions. On the x-axis, the range of political opinion. On the y-axis, the quantity of nodes with such political opinion.



(a) A network where nodes have moderate opinions and interact with each other.

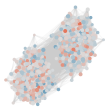
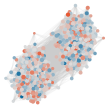
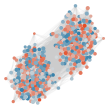
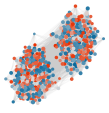
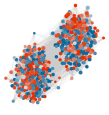
(b) A network where nodes have extreme opinions and interact less with each other.

Figure 7: From left to right, nodes tend to cluster in echo chambers and interact with nodes of similar opinion.

Thus, we focused on discovering how this operator performs better in directed networks, highlighting behaviours that can be studied only in directed networks. Therefore, we set up some experiments:

- An Asymmetric Stochastic Block Model, in which the k communities has the same probability p^{in} of co nnecting inside, but they connect outside with different probabilities $(p_1^{out}, ..., p_n^{out})$. In this case, one community connects outside with an higher probability than the other, like it can happens in hierarchical structures, where a few people influence a lot of people, but not the other way around. It has been tested with multiple θ values, to understand which is more useful for our case;
- A star graph, with a broadcaster/listener structure, where one single node, with an extreme opinion, interact with n nodes and vice versa;
- A troll army simulation, in which, generated a random graph [11] with modererate views, we add then a smaller group of trolls with extreme views interacting with the initial network.

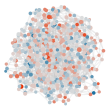
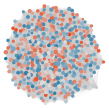
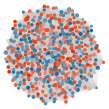
For each experiment, we performed some tests and compared the results obtained through the directed and undirected networks, which will be shown and discussed in Chapter 6.1. Encouraging and promising results have been found, that brought to the implementation of this measure on the Reddit dataset.

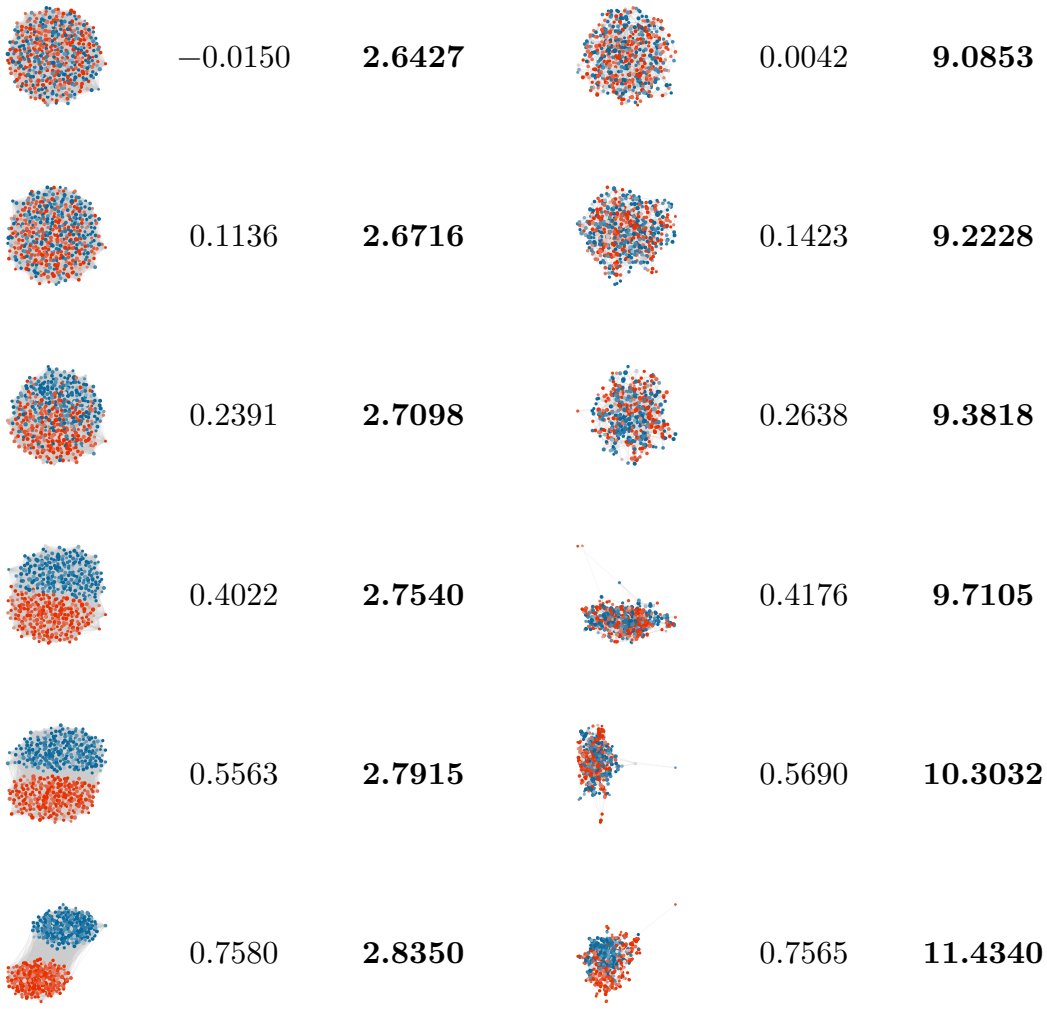
Network	Assortativity	Polarization	Network	Assortativity	Polarization
	−0.0066	**1.2128**		−0.0020	**2.1629**
	0.0072	**1.8880**		−0.0259	**3.4416**
	−0.0175	**3.0001**		−0.0351	**5.3296**
	0.0094	**4.1957**		−0.0115	**7.5942**
	−0.0007	**5.0249**		0.0023	**9.0245**

	0.1082	**5.2651**		0.1124	**9.1644**
	0.2330	**5.3801**		0.2429	**9.6671**
	0.3922	**5.5049**		0.4055	**10.0113**
	0.5448	**5.6235**		0.5556	**10.7486**
	0.7460	**5.7737**		0.7454	**11.5467**

(a) Stochastic Block model

(b) Barabási-Albert model

Network	Assortativity	Polarization	Network	Assortativity	Polarization
	0.0014	**0.7093**		−0.0336	**2.4097**
	−0.0078	**0.9391**		−0.0132	**3.3617**
	−0.0056	**1.5340**		0.0100	**5.5046**
	−0.0126	**2.2223**		−0.0026	**7.7785**



(c) Erdős-Rényi model (d) Watts-Strogatz model

Figure 8: From top to bottom, increasing extreme opinions (row 1 to 5) and increasing adversarial-node opinions isolation (row 6 ro 10).

5.2. Real networks implementation

After testing the toy examples, we moved on to the main project and implemented the results. Before implementing the new measure, support for directed networks was added by rewriting some sections of the pipeline. In addition to trivial changes regarding data structures and the removal of edge duplication to maintain the directed network (Code 2), the backboning section was redesigned.

By simply using directed data structures instead of undirected ones, the *thresholding* values in some weeks diverged from 2.625 (default value) to values $>$ 800. The problem occurred especially in weeks where less data than usual was available. Consequently, the final networks for those weeks contained very few nodes (on the order of tens, compared to networks with tens of thousands of nodes), invalidating the results.

The issue lies in the backboning section: it searches for the minimum threshold such that the ratio between the average degree of the network $\overline{k}$ and $\log_2(|V|)$ is less than 0.5, a heuristic that maximizes the number of nodes and minimizes the number of edges. To do this, it iteratively tests threshold values, counting how many edges and nodes survive and verifying whether the ratio $\frac{\overline{k}}{\log_2(|V|)}$ falls below the 0.5 threshold. The threshold criterion is given by:

$$\tilde{L}_{ij} - \text{threshold} \cdot \sqrt{var\left[\tilde{L}_{ij}\right]} > 0$$

The tested threshold and the new network, stripped of superfluous edges and nodes, are used to calculate the sparsity of the new network. Three situations can occur:

a) If all tested thresholds produce a sufficiently sparse network, the minimum threshold of 2.625 is returned; b) If there exists a threshold that produces a sparse network, the smallest of such thresholds is returned; c) If no threshold produces a sparse network, the algorithm is re-executed, increasing the window of tested threshold values.

It was precisely in this phase that the pipeline failed: in the directed case, the network contains about 40% more edges compared to the undirected projection (since $A \to B$ and $B \to A$ are distinct edges), causing a higher average degree. This made the ratio $\frac{\overline{k}}{\log_2(|V|)}$ exceed the 0.5 threshold for some weeks, triggering the iterative re-execution of the algorithm and causing the threshold to diverge to values greater than 800. At such thresholds, almost all edges were removed, producing very small networks.

The proposed solution consists of, *exclusively during the backboning phase*, symmetrizing the directed edges into an undirected projection, reducing the number of edges and stabilizing the ratio $\frac{\overline{k}}{\log_2(|V|)}$ below the 0.5 threshold for all weeks. Subsequently, only the edges whose node pairs passed the significance test are filtered from the original directed network, as shown in Code 1.

```
FUNCTION compute_directed_backbone(edges, is_directed):
  // For directed graphs, compute backbone on undirected projection
  // Filter original directed edges using undirected scores

  IF is_directed:
    // Create undirected projection by normalizing src/trg
    edges_undirected = COPY(edges)
    edges_undirected.src_norm = MIN(edges_undirected.src,
edges_undirected.trg)
    edges_undirected.trg_norm = MAX(edges_undirected.src,
edges_undirected.trg)

    // Aggregate edge weights for undirected pairs
    edges_undirected_agg = GROUP_BY(edges_undirected, [src_norm,
trg_norm]).SUM(nij)
```

```
    RENAME(edges_undirected_agg, src_norm → src, trg_norm → trg)

    // Double edges for undirected backbone calculation
    edges_undirected_agg = CONCAT(edges_undirected_agg,
                                  SWAP_SRC_TRG(edges_undirected_agg))

    // Compute undirected backbone
    edges_nc = bb.noise_corrected(edges_undirected_agg, undirected =
TRUE)
    threshold = find_bb_threshold(edges_nc, is_directed = FALSE)
    edges_nc_bb = bb.thresholding(edges_nc, threshold)

    // Map surviving undirected pairs to their scores
    pair_scores = DICT_FROM(edges_nc_bb, KEY=[src, trg], VALUE=score)

    // Filter original directed edges to surviving undirected pairs
    edges.src_norm = MIN(edges.src, edges.trg)
    edges.trg_norm = MAX(edges.src, edges.trg)
    edges.pair = ZIP(edges.src_norm, edges.trg_norm)

    surviving_pairs = SET_OF_PAIRS(edges_nc_bb, [src, trg])
    edges_filtered = FILTER(edges WHERE edges.pair IN surviving_pairs)

    // Assign undirected backbone scores to filtered directed edges
    edges_filtered.score = MAP(edges_filtered.pair, pair_scores)
    REMOVE_COLUMNS(edges_filtered, [pair, src_norm, trg_norm])

    // Create directed graph from filtered edges
    G = nx.from_pandas_edgelist(edges_filtered,
                                source="src",
                                target="trg",
                                edge_attr=TRUE,
                                create_using=nx.DiGraph())

  RETURN G
```

Code 1: Pseudocode of the new backboning function.

```
+ is_directed = sys.argv[1] == "directed"
+ is_signed = sys.argv[2] == "signed"

[...]
tensor = torch_geometric.data.Data(
+   edge_index = torch.tensor(np.array([edges["#src"].values,
edges["trg"].values]), dtype=torch.long).to(device)
-   edge_index = torch.tensor(np.array([pd.concat([edges["#src"],
edges["trg"]]).values, pd.concat([edges["trg"],
edges["#src"]]).values]), dtype=torch.long).to(device)
    node_vects =
torch.tensor(nodes.sort_values(by="node").set_index("node").drop("community",
axis=1).values, dtype=torch.float32).to(device)
+   edge_attr = torch.tensor(edges[["weight", "signific", "toxic",
"disagreement"]].values, dtype=torch.float32).to(device)
```

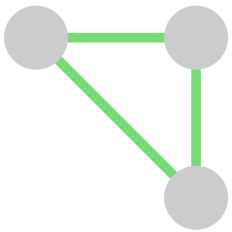
```
-   edge_attr = torch.tensor(pd.concat([edges[["weight", "signific",
"toxic", "disagreement"]], edges[["weight", "signific", "toxic",
"disagreement"]]]).values, dtype=torch.float32).to(device)
)

# Calculate the pseudoinverse of the Laplacian, this is the most
computationally intensive part so it's useful to cache it to re-use it
for all topics
- Linv = ps._Linv(tensor)
+ Linv = ps._Linv(tensor, mode="magnetic" IF is_directed ELSE
"classic", edges="signed" IF is_signed ELSE "unsigned")
```

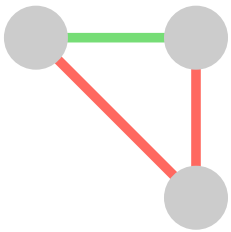
Code 2: File diffs to the file that evaluates and invert Laplacians.

Finally, after completing the migration of the source code, the pipeline was executed, and polarization was calculated, as seen in Chapter 2.3.10. Using the obtained results, measurements were performed on the network to gather correlations between the new results and the network's characteristics, comparing them with the results obtained from the previous undirected networks, including social balance[7], strengthening triangles (Figure 9(c)), repelling triangles (Figure 9(d))[8] and assortativity.

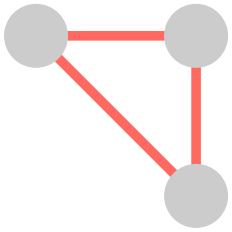
The polarization has been evaluated with four different values of $\theta = \left\{\frac{\pi}{2}, \frac{2}{3}\pi, \frac{4}{5}\pi, \pi\right\}$, to find which value has the highest effect in highlighting the polarization into the network. When $\theta = \pi$, it treat the network as a signed network [33].



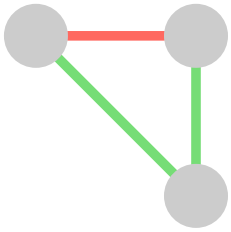
(a) Balanced triangle.



(b) Balanced triangle.



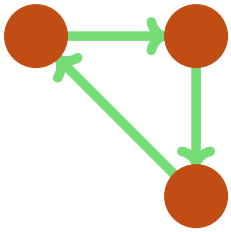
(c) Unbalanced triangle.



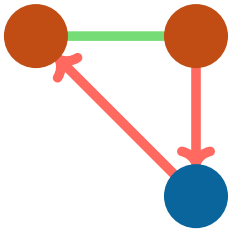
(d) Unbalanced triangle.

[7]Social balance is a measure that indicates the ratio between balanced triangles and all triangles in a network, based on toxicity (Figure 9(a) and Figure 9(b)).

[8]Similar to social balance, but they take into account the direction and political opinion of the nodes.



(e) Strengthening triangle.



(f) Repelling triangle.

Figure 9: Green edge: positive interaction, red edge: negative interaction.

In the next chapter will be analysed in detail the results obtained with the magnetic laplacian operator, both in random and real Reddit networks.

Results

In this chapter we are going to discuss the results obtained by the work done in the previous chapter. We will first analyse the results of the toy examples in Chapter 6.1 and then we are going to analyse the results obtained on the Reddit's networks dataset, in Chapter 6.2.

6.1. Toy Examples

In the first experiment, where we built an asymmetric SBM, we created some networks with the parameters in Table 2, with some interesting results that confirmed our starting assumptions.

#	n	k	p^{in}	p_A^{out}	p_B^{out}
1	$500, 500$	2	0.9	0.4	0.01
2	$50, 500$	2	0.9	0.4	0.01
3	$5, 500$	2	0.9	0.6	0.01

Table 2: Asymmetric SBMs and their parameters for each test.

We are spawning two communities, A and B, where their interactions are not symmetric. Indeed, the latter doesn't interact with the former (because of p_B^{out}), but the opposite is true. That is, A influences, but B doesn't. To A, we decide to iteratively decrease the number of nodes and slightly increase the interactions probability (p_A^{out}). In Figure 10, are shown the results for each test.

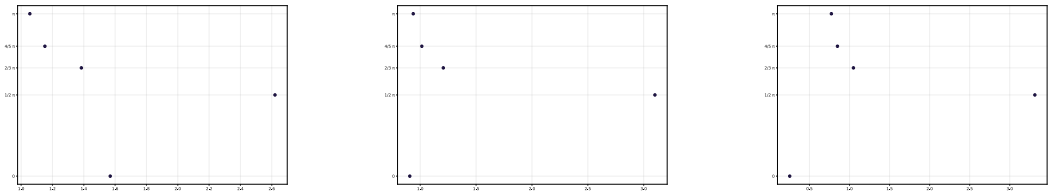


Figure 10: Different values of polarization with different θ values. x axis: polarization, y axis θ values.

When $\theta = 0$, the network is treated as undirected [33]. Is trivial to see that, in each test, the undirected counterpart is not sensitive to the directions.

But, it is interesting to see, that only when $\theta = \frac{\pi}{2}$, the directions are correctly highlighted. An answer is given by the fact that, as said in Chapter 2.4, $\theta = \frac{\pi}{2}$ is able to mostly catch 2, 3 and 4 length cycles. Inspecting the networks, this is confirmed by the fact that the number of squares (4 length cycles), occur with a greater frequency than the 2 and 3 length cycles, that are mostly captured by other θ values.

Specifically, when $\theta = 0$ and while reducing the size of community A, it is lead to a decrease of the polarization, close to 0, because the network is interpreted just as a network with a single community where all the nodes, overall, interact with each other, except for some noise (due to $|C_A| = 5$). On the contrary, when $\theta = \frac{\pi}{2}$, the polarization tends to grow because of the effect of the directed interactions

to the node vector distance. That is confirmed also by the number of cycles (especially the 4 length cycles) that tends to decrease in each test, even if the polarization grows (Figure 11).

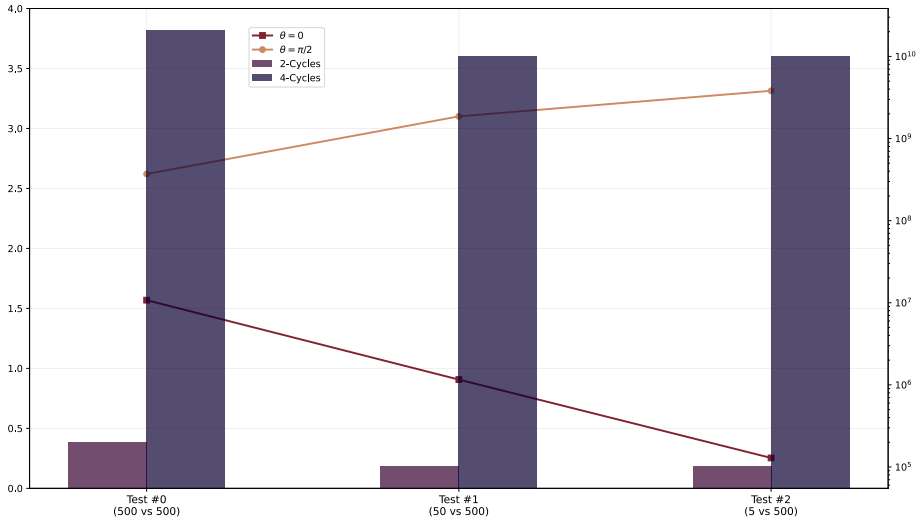


Figure 11: 2,4-length cycles and the undirected and directed polarization.

In the next experiment, we generated a star graph, that is a tree with one internal node (V_0) and k ($V_1, ..., V_k$) leaves. V_0 holds an extreme opinion -1 and the leaves hold the opposite extreme opinion, $+1$. The generation follows two patterns, the `broadcaster` and the `listener`. The `broadcaster` pattern, generate k edges that goes from V_0 to its neighbours: $V_0 \to \{V_1, ..., V_k\}$ and viceversa for the latter.

We performed several test (Table 3). Iterating, we changed the values of the opinions of the k nodes neighbours of V_0. What is interesting is that, from the topological side, the phenomenon we found out earlier, still remains, because the polarization tends to be near 0 when the network is undirected (is trivial here to note that if the network is undirected, there is no listener/broadcaster structure). However, there may be some limits that needs to be studied more:

- As can be seen in Table 3, the results are the same whether we are in lisener or broadcaster mode.

  This happens because reversing all the edges directions, is equivalent to the complex conjugate of the laplacian matrix: $(a + bi)^* = (a - bi)$. But, this means that the real part doesn't change.
- The results are also the same between the tests $\#1, \#2, \#5$ and $\#6$. This is due to the properties of the star graph, which is bipartite, and the behaviour of θ at $\frac{\pi}{2}$, that is a phase shift of 90°, that make the phenomenon make it work like a *Gauge transformation*, which is a physics phenomenon where the change of local variables, doesn't change the observable properties of the system. In fact, this behaviour can be avoided by not using a bipartite graph

or by using another value for θ. In tests #7 and #8, a demonstration with different θ values.

By this experiment, we can see that this operator is more sensitive, not to the opinions of the communities, but rather to the structural flow.

#	k	V_0	$V_{1\dots k}$	θ	mode	directed	result
0	50	1	-1	/	B/L	N	0.28
1	50	1	-1	$\frac{\pi}{2}$	B	Y	9.90
2	50	1	-1	$\frac{\pi}{2}$	L	Y	9.90
3	50	1	0	$\frac{\pi}{2}$	B	Y	0.20
4	50	1	0	$\frac{\pi}{2}$	L	Y	0.20
5	50	1	1	$\frac{\pi}{2}$	B	Y	9.90
6	50	1	1	$\frac{\pi}{2}$	L	Y	9.90
7	50	1	-1	$\frac{4}{5}\pi$	B	Y	17.12
8	50	1	1	$\frac{4}{5}\pi$	B	Y	16.56

Table 3: Parameters and results of the star graph experiment. B = `broadcaster`, L = `listener`.

In the last experiment, we tried to overcome to the limit we found in the previous experiment, where the bipartite graph didn't let us test the differences between the broadcaster and listeners. To do so, using the Erdős-Rényi model, we generated a random graph of n nodes. Following the same intuition behind the first experiment, to the nodes of the newly generated graph, we assigned an extreme value to the nodes: -1 and, then, we manually added a new set of m nodes. Those nodes, had the opposite opinion compared to the others, with a strong density among them. This models wants to simulate the phenomenon of the troll armies C_B, where a super dense community, with -often extremes- opinions, joins and starts to attack and annoy existing communities C_A, with opposite opinions than the former.

Later, we split the experiment in two tests: with the first, the smaller dense community C_B starts to undirectly engage with C_A, forming undirected edges inter-community $C_B \leftrightarrow C_A$). With the second experiment, the interactions are only one way: $C_B \rightarrow C_A$. As we can see from Table 4, the polarization is higher when the interactions are not mutual.

#	n	m	C_A	C_B	engaging	result
1	400	100	1	-1	N	2.04
2	400	100	1	-1	Y	2.49

Table 4: Parameters and results of the third experiment.

After having performed these experiments, we moved to the implementation on the real Reddit networks.

6.2. Real Networks

The fix proposed in Chapter 5.2, brought a directed polarization value shown in Figure 12, which is compared with the polarization evaluated with the undirected network.

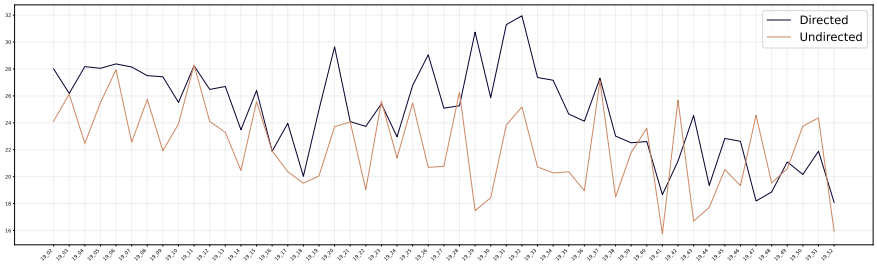
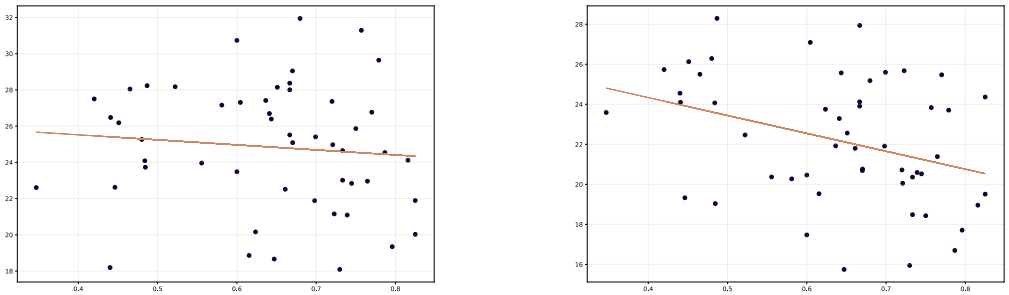


Figure 12: Polarization values (y-axis) through the year 2019 (x-axis) on Reddit networks.

Thus, the directed polarization has shown a stronger correlation with the strengthening triangles (Figure 14(a)) and the repulsing triangles (Figure 14(b)), compared to the undirected one (Figure 14(c) and Figure 14(d)). As found with the toy examples, this operator highlights the direction of the triangles (3-cycles), consequently catching more the positive interactions between likely-minded nodes. Undirected networks, because of their undirected nature, aren't able to do so. While correlation does not imply causality, these stronger relationships provide valuable empirical support for the idea that the directed measure captures structural nuances that the undirected measure misses.

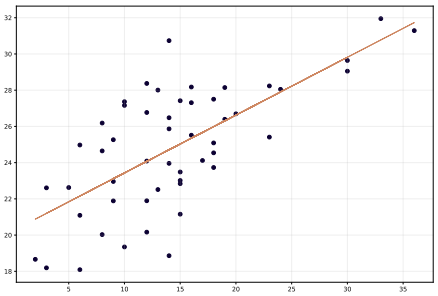
The social balance remains poorly correlated (Figure 13(a)), as in undirected networks (Figure 13(b)), but this is accepted, since it doesn't take into consideration the value of the nodes, but just the topology of the network.



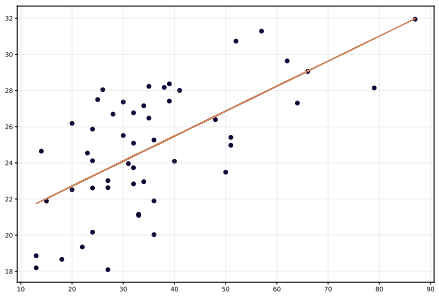
(a) *Directed* polarization (b) *Undirected* polarization

Figure 13: Scatter plot between social balance (x axis) and polarization (y axis).

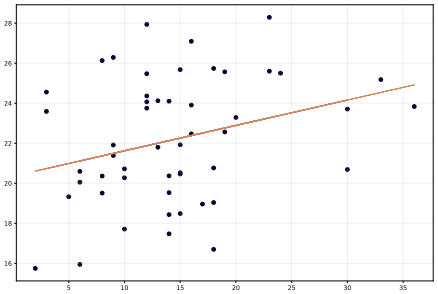
These results shows us that the node vector distance with the magnetic laplacian, doesn't only highlights the network structure, but highlights the value of each node, the stance in this case, which is essential to estimate how much a certain behaviour is spreading into the network.



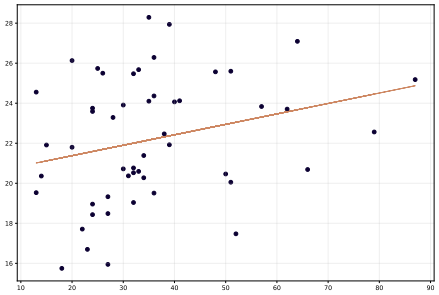
(a) *Strengthening* triangles, $corr = 0.67$



(b) *Repulsing* triangles, $corr = 0.64$



(c) *Strengthening* triangles, $corr = 0.31$



(d) *Repulsing* triangles, $corr = 0.26$

Figure 14: Scatter plot between number of triangles (x axis) and polarization (y axis). Top are evaluated on directed networks, bottom on undirected.

Conclusions and Future Work

In conclusion, throughout this thesis, we first introduced network science and the tools used in the field. We then discussed polarization, an important sociological phenomenon that reveals a great deal about interactions among actors in a network.

We then focused on the core of this work: node vector distance and the magnetic Laplacian. These are relatively new tools whose study has only just begun and must continue to better understand their properties and applications. In this thesis, we explored in particular how using these operators together can reveal new information in network analysis.

The results obtained with the toy examples, in Chapter 6.1, showed that preserving edge direction in a network can uncover information that would otherwise remain hidden, such as small communities influencing larger ones. In a world where interactions are becoming more asymmetrical, having the tools to study and quantify these behaviours is useful for maintaining the scientific rigor needed. Although the higher correlations observed do not establish causal superiority, they serve as a promising indicator that the new operator highlight previously hidden structures.

However, this work is far from complete. The results reminded us that understanding and adding complexity to an already complex problem is not easy to solve. There are still problems we have not tackled that would be both interesting and useful to address. A future extension of this work should approach this topic from a physics and mathematical perspective, which we could not do because we lacked the necessary background, by testing this operator not only empirically but also formally. The open questions we could not answer, but that I am sure future work will address, are the following: should node vector distance (polarization) in a network where all nodes are extreme but share the same stance be as high as in a network where there is more tension between nodes because they hold opposite stances? Why? This occurred in the second and third experiments as seen in Chapter 6.1.

One lesson I personally learned from this experience in network science, but that also applies to most (if not all) scientific problems, is that there is no one-size-fits-all solution. On the contrary, each complex problem has many variables and peculiarities. The best solution maximizes or minimizes the return value of the function, but it depends on all the arguments we choose to include and on the constraints we face.

Likewise, the polarization measure does not have a single best θ value to use in every case. It must be carefully fine-tuned based on the structures, cycles, and directions we want to highlight.

In the end, I found myself more interested in understanding how the operator works than in applying it to the original Reddit dataset, which was the initial premise of the internship.

Approaching problems from a theoretical perspective means that studying new operators is the end goal, while applying them to real-world data is the means to test whether what you study is useful or not; this is the opposite of starting only from an operator or technique to infer answers about real-world behaviours and phenomena.

Personally, I started this experience with many questions and ended it with even more questions than at the beginning, both about the problem I worked on during the internship and, more generally, about my academic and professional career.

Closing Notes

The source code of this document and the data for the generation of the figures above, can be found on GitHub, on my public profile[9].

8.1. Disclaimer

For the writing of this thesis, no generative AI was used, except for assistance with syntactical, grammatical, and structural adjustments.Generative AI has been used to help writing some functions for the analysis of the data, refactoring the codebase and brainstorming sessions.

8.2. Funding Acknowledgments

The internship thanks to whom this thesis has been possible, was partially funded by *Erasmus+ Traineeship Scholarship* and by Danish Data Science Academy (DDSA), under the grant *Visit Grant*. I appreciate their financial support and the opportunities it provided.

8.3. Acknowledgments

Here I am, writing the part of this thesis that I postponed the most. If someone had asked me four years ago to bet on this moment, I would have missed the opportunity to be a millionaire. These past *3.75* years have been full of experiences that would never have happened if I had not chosen to begin this journey. And now, here I am. It may sound cliché, but a lesson I learned along the way is that the people we choose and those we meet shape our experiences, for better and for worse. I feel very fortunate for having crossed paths with so many people who made this experience memorable. I hope I have, in return, been a good colleague, friend, partner, relative, and son. In no particular order, I want to personally thank:

- My ITU supervisor, the professor Michele Coscia; who invited me in his research lab and treated me as a peer, even though I was just a bachelor student. Thank you for your constant support and rigorous feedback throughout this project. Your advices always gave me new perspectives to think on and that improved the quality of this work;
- My UniMi supervisor, the professor Elena Casiraghi, who has been always willing to help and to correcting this thesis. Thank you for your quick yet detailed feedback on my work and your dedication;

[9] https://github.com/demic-dev/bsc-thesis

- The NERDS group, which, alongside prof. Coscia, made me feel very welcome in Copenhagen, helping me in any way you can. I have met wonderful and talented people and gave me a wonderful perspective of the academic world;
- UniMi and Regione Lombardia, that with their scolarship helped me to live this experience by focusing mainly on my studies.

Beyond the academic support, I am deeply grateful for the love and support from my girlfriend, my friends and my family. Your support and presence in my life means a lot to me.

I am sure I will forget someone, so I will make sure to express my gratitude to each of you privately. A big and heartfelt thank you.

Bibliography

[1] A. Sylolypavan, D. Sleeman, H. Wu, and M. Sim, The impact of inconsistent human annotations on AI driven clinical decision making, NPJ Digit. Med. **6**, 26 (2023).

[2] M. Coscia, *The Atlas for the Aspiring Network Scientist* (2021).

[3] D. G. Myers and H. Lamm, The group polarization phenomenon., Psychological Bulletin **83**, 602 (1976).

[4] T. Tichelbaecker, N. Gidron, W. Horne, and J. Adams, What Do We Measure When We Measure Affective Polarization across Countries?, Public Opinion Quarterly **87**, 803 (2023).

[5] M. Hohmann and M. Coscia, Estimating affective polarization on a social network, Plos One **20**, e328210 (2025).

[6] J. L. Peterson and A. Silberschatz, *Operating System Concepts*, 2nd ed. (Addison-Wesley, 1985).

[7] L. R. Ford, Network Flow Theory, technical report, 1956.

[8] E. W. Dijkstra, A Note on Two Problems in Connexion with Graphs, Numerische Mathematik **1**, 269 (1959).

[9] N. R. Council, *Network Science* (The National Academies Press, Washington, DC, 2005).

[10] A.-L. Barabási and R. Albert, Emergence of Scaling in Random Networks, Science **286**, 509 (1999).

[11] P. L. Erdős and A. Rényi, On random graphs. I., Publicationes Mathematicae Debrecen (2022).

[12] M. Coscia, A. Gomez-Lievano, J. Mcnerney, and F. Neffke, The Node Vector Distance Problem in Complex Networks, ACM Comput. Surv. **53**, (2020).

[13] P. G. Doyle and J. L. Snell, Random Walks and Electric Networks, (2000).

[14] G. Kirchhoff, On the Solution of the Equations Obtained from the Investigation of the Linear Distribution of Galvanic Currents, IRE Transactions on Circuit Theory **5**, 4 (1958).

[15] F. Váša and B. Mišić, Null models in network neuroscience, Nature Reviews Neuroscience **23**, 493 (2022).

[16] P. W. Holland, K. B. Laskey, and S. Leinhardt, Stochastic blockmodels: First steps, Social Networks **5**, 109 (1983).

[17] D. J. Watts and S. H. Strogatz, Collective dynamics of `small-world' networks, Nature **393**, 440 (1998).

[18] M. Á. Serrano, M. Boguñá, and A. Vespignani, Extracting the multiscale backbone of complex weighted networks, Proceedings of the National Academy of Sciences **106**, 6483 (2009).

[19] D. Grady, C. Thiemann, and D. Brockmann, Robust classification of salient links in complex networks, Nature Communications **3**, 864 (2012).

[20] M. Coscia and F. M. Neffke, *Network Backboning with Noisy Data*, in *2017 IEEE 33rd International Conference on Data Engineering (ICDE)* (2017), pp. 425–436.

[21] H. S. Wilf, The Eigenvalues of a Graph and Its Chromatic Number, Journal of the London Mathematical Society 330 (1967).

[22] D. A. Spielman, *Spectral and Algebraic Graph Theory* (2025).

[23] K. M. Hall, An r-Dimensional Quadratic Placement Algorithm, Management Science **17**, 219 (1970).

[24] M. E. J. Newman, Equivalence between modularity optimization and maximum likelihood methods for community detection, Physical Review E **94**, (2016).

[25] M. Rosvall, D. Axelsson, and C. T. Bergstrom, The map equation, The European Physical Journal Special Topics **178**, 13 (2009).

[26] Wikipedia, *Codifica Di Huffman — Wikipedia, L'enciclopedia Libera*, http://it.wikipedia.org/w/index.php?title=Codifica_di_Huffman&oldid=147328281.

[27] U. N. Raghavan, R. Albert, and S. Kumara, Near linear time algorithm to detect community structures in large-scale networks, Physical Review E **76**, (2007).

[28] D. Chakrabarti, R. Kumar, and A. Tomkins, *Evolutionary Clustering*, in *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Association for Computing Machinery, Philadelphia, PA, USA, 2006), pp. 554–560.

[29] M. Coscia, A. Gomez-Lievano, J. Mcnerney, and F. Neffke, The Node Vector Distance Problem in Complex Networks, ACM Comput. Surv. **53**, (2020).

[30] M. Hohmann, K. Devriendt, and M. Coscia, Quantifying ideological polarization on a network using generalized Euclidean distance, Science Advances **9**, eabq2044 (2023).

[31] M. A. Shubin, Discrete magnetic Laplacian, Communications in Mathematical Physics **164**, 259 (1994).

[32] D. Krejcirik, N. Raymond, and M. Tusek, *The Magnetic Laplacian in Shrinking Tubular Neighbourhoods of Hypersurfaces*, https://arxiv.org/abs/1303.4753.

[33] M. Fanuel, C. M. Alaíz, and J. A. K. Suykens, Magnetic eigenmaps for community detection in directed networks, Physical Review E **95**, (2017).

[34] J. S. Fabila-Carrasco, F. Lledó, and O. Post, Matching number, Hamiltonian graphs and magnetic Laplacian matrices, Linear Algebra and Its Applications **642**, 86 (2022).

[35] G. Van Rossum and F. L. Drake Jr, *Python Tutorial* (Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995).

[36] A. A. Hagberg, D. A. Schult, and P. J. Swart, *Exploring Network Structure, Dynamics, And Function Using Networkx*, in *Proceedings of the 7th Python in Science Conference*, edited by G. Varoquaux, T. Vaught, and J. Millman (Pasadena, CA USA, 2008), pp. 11–15.

[37] C. R. Harris et al., Array programming with NumPy, Nature **585**, 357 (2020).

[38] J. Ansel et al., *PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation*, in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)* (ACM, 2024).

[39] The pandas development team, *pandas-dev/pandas: Pandas*, https://doi.org/10.5281/zenodo.3509134.

[40] Similarweb, *Top Websites Ranking*, https://www.similarweb.com/top-websites/.

[41] R. stuck_in_the_matrix Watchful1, Reddit comments/submissions 2005-06 to 2025-12, (2025).

[42] M. Grootendorst, BERTopic: Neural topic modeling with a class-based TF-IDF procedure, Arxiv Preprint Arxiv:2203.05794 (2022).

[43] L. Hanu and Unitary team, Detoxify, (2020).

[44] AI@Meta, Llama 3 Model Card, (2024).

[45] E. Rossi, B. Charpentier, F. D. Giovanni, F. Frasca, S. Günnemann, and M. Bronstein, *Edge Directionality Improves Learning on Heterophilic Graphs*, https://arxiv.org/abs/2305.10498.

[46] H. Sun, X. Li, D. Su, J. Han, R.-H. Li, and G. Wang, *Towards Data-Centric Machine Learning on Directed Graphs: A Survey*, https://arxiv.org/abs/2412.01849.

[47] E. Kummerfeld, A simple interpretation of undirected edges in essential graphs is wrong, Plos One **16**, e249415 (2021).

[48] O. Sporns, Graph theory methods: applications in brain networks, Dialogues Clin. Neurosci. **20**, 111 (2018).